

Stateless vs Stateful Widget

Understanding StatelessWidget and StatefulWidget

- a. What are StatelessWidget and StatefulWidget?
 - Flutter has two main types of widgets: StatelessWidget and StatefulWidget.
 - Both are used to build UI but behave differently.
 - Stateless widgets do not change after being built, while Stateful widgets can change dynamically.
- b. What is a StatelessWidget?
 - A StatelessWidget does not change once it is built.
 - It stays the same unless Flutter explicitly rebuilds it.
 - Best for displaying static content (e.g., text, icons, images).

```
class TitleWidget extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Text('Welcome to My App');  
  }  
}
```

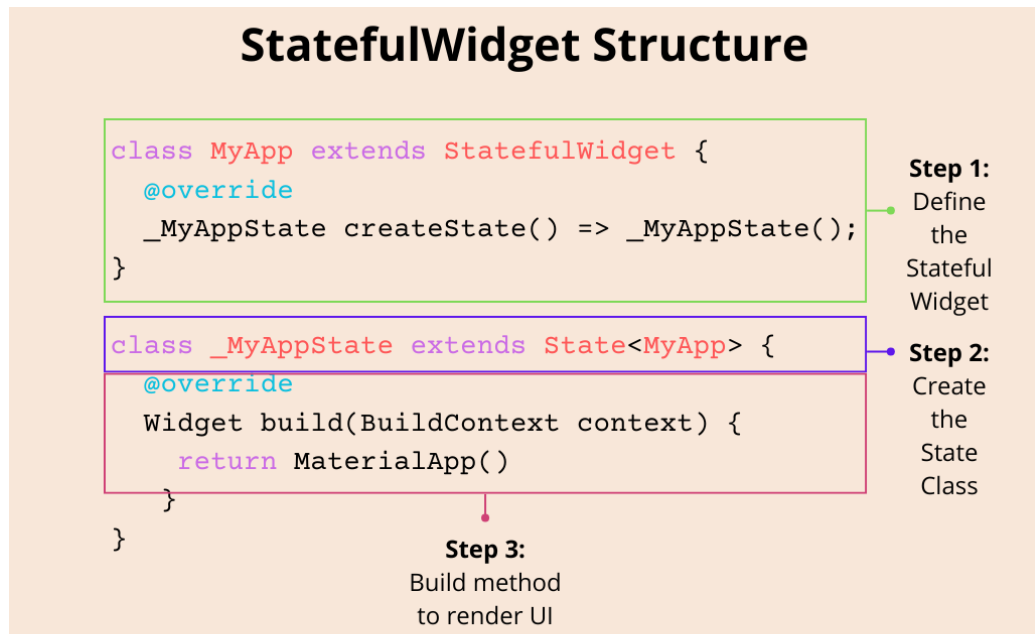
- Explanation:
 - The widget always shows “Welcome to My App”.
 - No matter what happens in the app, this text does not change.
- c. What is a StatefulWidget?
 - A StatefulWidget is used when a widget needs to update its UI based on interaction or data changes.
 - It is useful for cases where the UI does not stay static.

```
class ButtonWidget extends StatefulWidget {  
  @override  
  _ButtonWidgetState createState() => _ButtonWidgetState();  
}  
  
class _ButtonWidgetState extends State<ButtonWidget> {  
  String buttonText = 'Press me';  
  
  void updateText() {  
    setState(() {  
      buttonText = 'You pressed the button!';  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return ElevatedButton(  
      onPressed: updateText,  
      child: Text(buttonText),  
    );  
  }  
}
```

```
}

```

- Explanation:
 - Initially, the button shows “Press me”.
 - When the button is clicked, setState() updates the text.
 - Because it is a StatefulWidget, the UI changes dynamically.
- d. Structure of a StatefulWidget



- Breaking Down the Structure :

Step	What Happens	Purpose	Example
1	Define a class that extends a special widget type	Enables stateful behavior	class ButtonWidget extends StatefulWidget
2	Provide a method to connect it with a logic controller	Returns the object that holds state	@override _ButtonWidgetState createState() => _ButtonWidgetState();
3	Create a second class for managing changes	This class determines how the widget reacts	class _ButtonWidgetState extends State<ButtonWidget> {}
4	Use a special method to trigger UI updates	This notifies Flutter to rebuild the widget	setState(() { buttonText = 'Updated!'; });

- ◇ In the widget class, you'll see a method that is redefined from the superclass.
 - ◇ The return value of that method is an instance of another class that handles the logic.
 - ◇ That logic-handling class always extends another class with a type argument that refers back to the widget.
 - ◇ The build structure is declared inside the logic-handling class using a method that constructs and returns widgets.
- e. Differences Between StatelessWidget and StatefulWidget

▪ Comparison Table :

Feature	StatelessWidget	StatefulWidget
Changes Over Time	No	Yes
State Management	No internal state	Has internal state
Performance	More efficient	Less efficient
Example	Static text, icons, images	Interactive buttons, forms
Widget Class	class MyWidget extends StatelessWidget	class MyWidget extends StatefulWidget
Build Method	Only needs build method	Needs createState and State class

f. Comparison with Java (Android Development)

- In Java (Android), UI components are updated using View and Listener.

```
TextView textView = findViewById(R.id.myTextView);
textView.setText("Welcome to My App");
```

```
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        button.setText("You pressed the button!");
    }
});
```

- In Flutter, UI updates are handled declaratively using setState().

Understanding setState in Flutter

a. What is setState()?

- setState notifies Flutter that something has changed in the widget, triggering a UI rebuild.
- It takes a function where you modify the widget's state.
- Commonly used for handling user interactions like button clicks, form input, and animations.
- This is an example counter application :

```
class CounterWidget extends StatefulWidget {
    @override
    _CounterWidgetState createState() =>
    _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
    int counter = 0;

    void incrementCounter() {
        setState(() {
            counter++; // Update the counter
        });
    }

    @override
```